

GITHUB ACTIONS - LEARNING NOTES

29/07/2025

* What is GitHub Actions?

GitHub Actions is a CI/CD platform built into GitHub. It helps us automate build, test and deployment pipelines directly in our repositories.

WORKFLOW FILE LOCATION

Workflow files are written in YAML and stored in:

• `github/workflows/`

* KEY CONCEPTS

① Workflow

A workflow is an automated process defined in a YAML file.

② Event (Triggers)

Workflows run when a specific event happens in the repository.

Examples:

- push
- pull_request
- release
- workflow_dispatch
- schedule (cron)

③ Job

A workflow contains one or more jobs.

Jobs run in parallel by default.

Each job runs on a fresh runner.

④ Runner

Runner is a server that executes your workflow.

GitHub hosted runners:

- ubuntu-latest
- windows-latest
- macos-latest

We can also use Self-hosted runners.

⑤ Steps

Each job has multiple steps.

Steps run sequentially in a job.

Example steps: checkout code, install dependencies, run tests, build, deploy.

⑥ Action

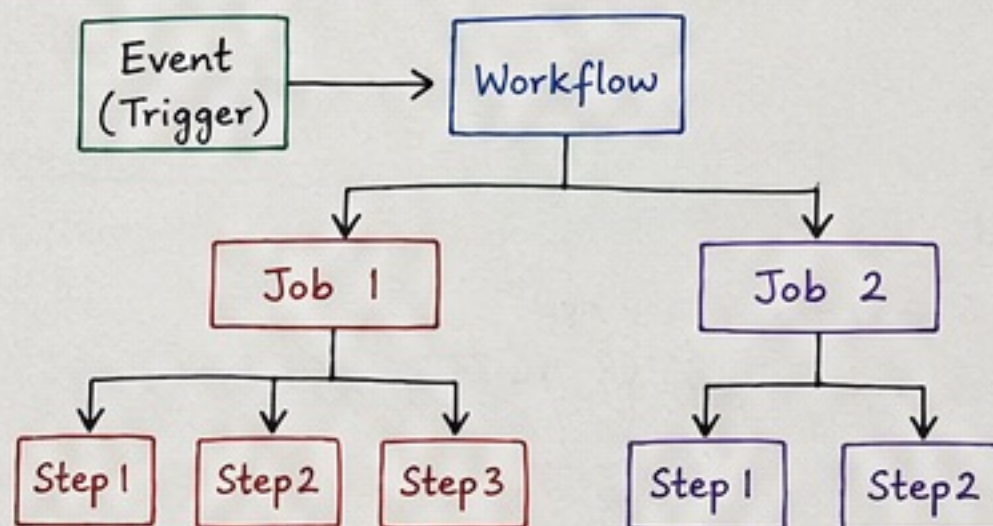
Actions are reusable units of code that perform a task.

We can use ready-made actions from GitHub Marketplace or create custom actions.

SIMPLE WORKFLOW EXAMPLE

```
name: Simple CI
on:
  push:
    branches: [ main ]    # Runs on push to main
jobs:
  build:                  # Job id
    runs-on: ubuntu-latest # Runner
    steps:                 # List of steps
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - name: Install dependencies
        run: pip install -r requirements.txt
      - name: Run Tests
        run: pytest
```

WORKFLOW STRUCTURE



ENVIRONMENT VARIABLES

env:

APP_ENV: production
DEBUG: 'false'

Use as → `${{ env.APP_ENV }}`

SECRETS

Store sensitive data in repository Secrets.

Repo → Settings → Secrets and variables → Actions

Use as → `${{ secrets.MY_SECRET }}`

MATRIX STRATEGY

strategy:

matrix:

python-version: ['3.10', '3.11', '3.12']

Runs the job for multiple combinations.

ARTIFACTS

- Used to store files generated during workflow run.
- Can be downloaded and used in another job or workflow.

BENEFITS

- ✓ Native GitHub integration
- ✓ Easy to set up and use
- ✓ Automates repetitive tasks
- ✓ Supports CI/CD pipelines
- ✓ Reusable workflows and actions
- ✓ Cross-platform runners
- ✓ Large marketplace of actions
- ✓ Improves speed and quality of delivery

IMPORTANT NOTES

- Workflows are written in YAML.
- File must end with .yml or .yaml
- Indentation is important (use spaces).
- Use meaningful job and step names.
- Keep secrets safe, never hardcode.
- Test workflows in feature branches first.

FILE EXTENSION

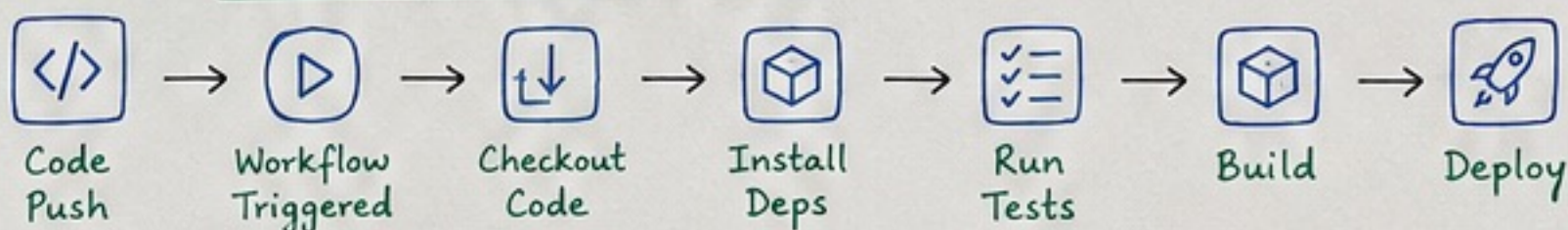
- .yml (recommended)
- .yaml

Both are valid.

Example: ci.yml
deploy.yaml



TYPICAL CI PIPELINE FLOW





≡ Understanding a Workflow (Line by Line) ≡

1. What is a Workflow?

- A workflow is an automated process that you define in a YAML file.
- It runs on GitHub hosted runners when a specific event happens.

Where is the workflow file?

.github/workflows/
example: ci.yml

2. Lifecycle of a Workflow

- Event is triggered (push, PR, etc.)
- GitHub reads the workflow file
- Jobs are created
- Runner is allocated
- Steps are executed in order
- Workflow completes (success/failure)



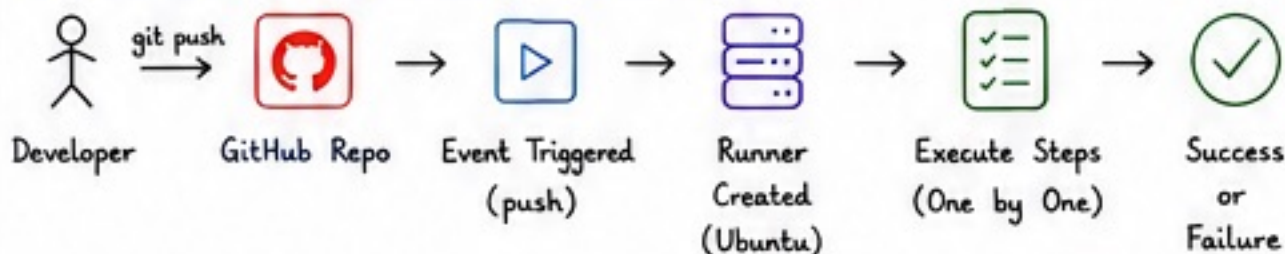
3. Example - Python CI Workflow

```
1 name: Python CI
2 on:
3   push:
4     branches:
5       - main
6 jobs:
7   build:
8     runs-on: ubuntu-latest
9     steps:
10    - name: Checkout Repository
11      uses: actions/checkout@v4
12    - name: Setup Python
13      uses: actions/setup-python@v5
14      with:
15        python-version: "3.12"
16    - name: Install Dependencies
17      run: pip install -r requirements.txt
18    - name: Run Tests
19      run: pytest
```

Quick Tip

- ✓ Indentation is very important in YAML.
- ✓ Use 2 spaces for each level.
- ✓ Always start with actions/checkout.
- ✓ Pin action versions like @v4, @v5.

4. Workflow Execution Flow (High Level)



5. Explain Every Statement (Line by Line)

#	Statement	Purpose	Why do we need it?	Example / Notes
①	name	Gives a name to the workflow.	Helps identify the workflow in Actions tab.	name: Python CI
②	on	Defines the event that triggers the workflow.	Tells GitHub when to start this workflow.	on: push
③	push	Runs when code is pushed.	Automates workflow on every push.	push: branches: - main
④	branches	Specifies which branches trigger the workflow.	Avoids running workflow for unwanted branches.	branches: - main
⑤	jobs	Contains one or more jobs.	Workflow is divided into independent jobs.	jobs: build:
⑥	build	Job ID (name).	Helps reference the job (if multiple jobs).	build:
⑦	runs-on	Specifies the runner / machine.	Decides the OS on which the job runs.	runs-on: ubuntu-latest
⑧	steps	List of tasks executed.	Breaks the job into sequential steps.	steps:
⑨	uses	Uses a pre-built GitHub Action.	Reuses community actions to save time.	uses: actions/ checkout@v4
⑩	with	Passes inputs / parameters to the action.	Customizes the action behavior.	with: python-version: "3.12"
⑪	run	Executes shell commands.	Runs our own commands on the runner.	run: pip install -r requirements.txt
⑫	python-version	Installs the specified Python version.	Ensures consistent Python version.	python-version: "3.12"

Common Mistakes

- ✗ Wrong indentation in YAML.
- ✗ Forgetting actions/checkout.
- ✗ Incorrect Python version.
- ✗ Wrong branch name / triggers.
- ✗ Missing requirements.txt.

Interview Tips

- Workflow → Complete automation process.
- Job → Collection of steps.
- Step → Single task.
- Action → Reusable code.
- Runner → Machine executing the job.

6. How Everything Works Together (Detailed Flow)



7. Common Triggers (Events)



8. Key Differences

Concept	Difference
Workflow vs Job	Workflow = Entire automation. Job = Part of workflow.
Job vs Step	Job = Group of steps that run on same runner. Step = Single task in a job.
uses vs run	uses = Reusable action written by others. run = Our own shell command.
Runner vs Agent	Runner = GitHub hosted machine. (Agent is a generic term; in GitHub Actions we say Runner.)

9. Best Practices

- ★ Use meaningful names for workflows, jobs and steps.
- ★ Keep jobs small and focused.
- ★ Pin action versions (use @v4, @v5, not @main).
- ★ Store secrets in GitHub Secrets.
- ★ Use environment variables for configuration.
- ★ Always test workflows in feature branches first.

10. Interview Questions (Must Know!)

- What is GitHub Actions?
- What is the difference between workflow and job?
- What is the difference between step and action?
- What is the difference between uses and run?
- Why is actions/checkout necessary?
- What happens if one step fails?
- What is a runner?
- How do you trigger a workflow manually?

Remember

Automation saves time, reduces human errors and makes our deployments reliable! ☺